

# Inventory-Deus

Table Connections — How Primary Keys and Foreign Keys wire 40 tables across 16 services into a single system.

## Part 1 — The principles

### 1.1 What a primary key is

Every row in every table is uniquely identified by a single column called the **primary key** (PK). In this system the PK is always named `id` and is a `BIGSERIAL` — an auto-incrementing 64-bit integer. The database guarantees two things about it: it is never null, and it is never duplicated within that table.

You can think of a PK as the row's permanent address. Once `catalog.sku(id=501)` exists, any other row anywhere in the database can point at it by saying "I belong to sku 501" and the database will keep that link honest forever.

### 1.2 What a foreign key is

A **foreign key** (FK) is a column in one table that holds the PK value of a row in another table. It is the wire that connects them. When you declare a column as a foreign key, Postgres refuses to accept any value in it that does not exist as a PK in the referenced table. It also refuses to delete the referenced row while children still point at it (unless you explicitly allow cascades).

A foreign key column is conventionally named after the table it points at, with an `_id` suffix. So `procurement.po_line.sku_id` points at `catalog.sku.id`. The name tells you the relationship before you even open the schema.

### 1.3 The three relationship shapes

#### One-to-many (the most common shape in this system)

A purchase order has many lines. The *many* side holds the foreign key.

|                                         |                        |                                  |
|-----------------------------------------|------------------------|----------------------------------|
| <code>procurement.purchase_order</code> |                        | <code>procurement.po_line</code> |
| <code>id (PK) = 44</code>               | <code>&lt;-----</code> | <code>po_id (FK) = 44</code>     |
|                                         |                        | <code>po_id (FK) = 44</code>     |
|                                         |                        | <code>po_id (FK) = 44</code>     |

#### Many-to-one across schemas

Many `po_line` rows in the *procurement* schema can reference the same `sku` row in the *catalog* schema. This is what makes the SKU a "hub" entity.

|                            |                        |                                  |
|----------------------------|------------------------|----------------------------------|
| <code>catalog.sku</code>   |                        | <code>procurement.po_line</code> |
| <code>id (PK) = 501</code> | <code>&lt;-----</code> | <code>sku_id (FK) = 501</code>   |
|                            |                        | <code>sku_id (FK) = 501</code>   |
|                            |                        | <code>sku_id (FK) = 501</code>   |

#### Many-to-many (needs a junction table)

A user can have many roles; a role can be held by many users. Neither side can hold the FK alone — you need a third table that holds two FKs.

|                            |                     |                               |                     |                          |
|----------------------------|---------------------|-------------------------------|---------------------|--------------------------|
| <code>auth.app_user</code> |                     | <code>auth.user_role</code>   |                     | <code>auth.role</code>   |
| <code>id (PK) = 7</code>   | <code>&lt;--</code> | <code>user_id (FK) = 7</code> |                     |                          |
|                            |                     | <code>role_id (FK) = 2</code> | <code>--&gt;</code> | <code>id (PK) = 2</code> |

### 1.4 Cross-schema foreign keys

Because the entire system lives in a single Postgres database (with one schema per service), foreign keys are allowed to point across schemas. This is the single most important property of the design:

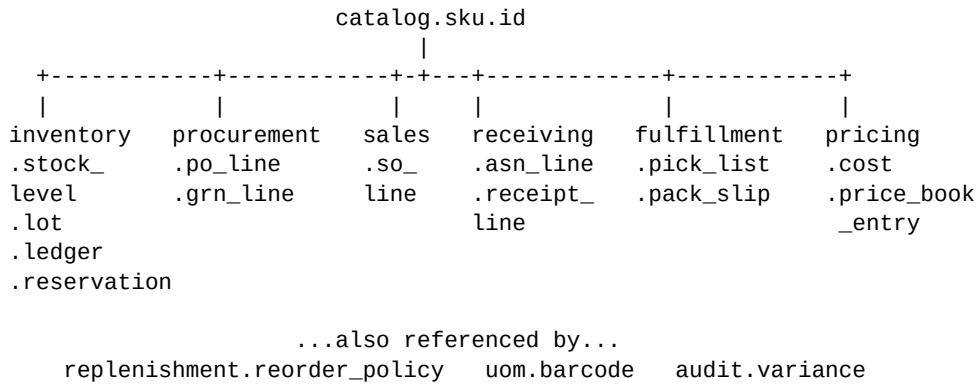
`procurement.po_line.sku_id` can reference `catalog.sku.id` directly, and Postgres enforces that link at every INSERT, UPDATE, and DELETE. If catalog and procurement were in separate databases this would be impossible — you would only have HTTP calls and prayers.

## Part 2 — The master connection map

Two entities are the "hubs" that almost every other table points at. Understand these two and the rest of the system falls into place.

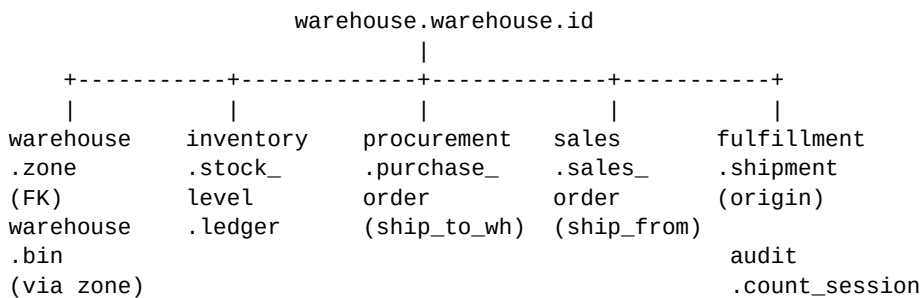
### 2.1 The SKU hub

`catalog.sku.id` is the most-referenced primary key in the system. Every operational table that records "what" eventually points back at it.



### 2.2 The Warehouse hub

`warehouse.warehouse.id` is the second hub. "Where" is part of almost every operational row.



### 2.3 Why two hubs?

Inventory is fundamentally the intersection of *what* and *where*. Every operational table can be read as "a thing happening to SKU X at Warehouse Y". The two-hub design just makes that explicit in the schema.

## Part 3 — Every table, every key

For each service, this section lists every table with its primary key, then every foreign key it carries (with the target table in parentheses). FKs that cross into another schema are shown with their schema prefix.

### Product / Catalog Service (schema: catalog)

| Table            | PK | Foreign keys → target                                     |
|------------------|----|-----------------------------------------------------------|
| <b>product</b>   | id | category_id → catalog.category(id)                        |
| <b>sku</b>       | id | product_id → catalog.product(id)<br>uom_id → uom.unit(id) |
| <b>variant</b>   | id | product_id → catalog.product(id)                          |
| <b>category</b>  | id | parent_id → catalog.category(id) – self-ref for tree      |
| <b>attribute</b> | id | product_id → catalog.product(id)                          |

### Inventory Service (schema: inventory)

| Table               | PK | Foreign keys → target                                                                                                                                         |
|---------------------|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>stock_level</b>  | id | sku_id → catalog.sku(id)<br>warehouse_id → warehouse.warehouse(id)<br>bin_id → warehouse.bin(id)                                                              |
| <b>lot</b>          | id | sku_id → catalog.sku(id)                                                                                                                                      |
| <b>serial</b>       | id | sku_id → catalog.sku(id)<br>lot_id → inventory.lot(id)                                                                                                        |
| <b>reservation</b>  | id | sku_id → catalog.sku(id)<br>warehouse_id → warehouse.warehouse(id)<br>so_line_id → sales.so_line(id)                                                          |
| <b>stock_ledger</b> | id | sku_id → catalog.sku(id)<br>warehouse_id → warehouse.warehouse(id)<br>bin_id → warehouse.bin(id)<br>lot_id → inventory.lot(id)<br>user_id → auth.app_user(id) |

### Warehouse Service (schema: warehouse)

| Table                   | PK | Foreign keys → target                           |
|-------------------------|----|-------------------------------------------------|
| <b>warehouse</b>        | id | <i>(no foreign keys — this is a root table)</i> |
| <b>zone</b>             | id | warehouse_id → warehouse.warehouse(id)          |
| <b>bin</b>              | id | zone_id → warehouse.zone(id)                    |
| <b>capacity</b>         | id | bin_id → warehouse.bin(id)                      |
| <b>picking_strategy</b> | id | warehouse_id → warehouse.warehouse(id)          |

## Supplier Service (schema: supplier)

| Table                   | PK | Foreign keys → target                                               |
|-------------------------|----|---------------------------------------------------------------------|
| <b>supplier</b>         | id | (no foreign keys — this is a root table)                            |
| <b>contact</b>          | id | supplier_id → supplier.supplier(id)                                 |
| <b>lead_time</b>        | id | supplier_id → supplier.supplier(id)<br>sku_id → catalog.sku(id)     |
| <b>price_list</b>       | id | supplier_id → supplier.supplier(id)                                 |
| <b>price_list_entry</b> | id | price_list_id → supplier.price_list(id)<br>sku_id → catalog.sku(id) |
| <b>terms</b>            | id | supplier_id → supplier.supplier(id)                                 |

## Procurement / Purchase Service (schema: procurement)

| Table                 | PK | Foreign keys → target                                                                                                          |
|-----------------------|----|--------------------------------------------------------------------------------------------------------------------------------|
| <b>purchase_order</b> | id | supplier_id → supplier.supplier(id)<br>warehouse_id → warehouse.warehouse(id) – ship-to<br>created_by → auth.app_user(id)      |
| <b>po_line</b>        | id | po_id → procurement.purchase_order(id)<br>sku_id → catalog.sku(id)<br>uom_id → uom.unit(id)                                    |
| <b>grn</b>            | id | po_id → procurement.purchase_order(id)<br>warehouse_id → warehouse.warehouse(id)                                               |
| <b>grn_line</b>       | id | grn_id → procurement.grn(id)<br>po_line_id → procurement.po_line(id)<br>sku_id → catalog.sku(id)<br>lot_id → inventory.lot(id) |
| <b>invoice</b>        | id | po_id → procurement.purchase_order(id)<br>supplier_id → supplier.supplier(id)                                                  |
| <b>approval</b>       | id | po_id → procurement.purchase_order(id)<br>approver_id → auth.app_user(id)                                                      |

## Sales Order Service (schema: sales)

| Table              | PK | Foreign keys → target                                                                      |
|--------------------|----|--------------------------------------------------------------------------------------------|
| <b>sales_order</b> | id | warehouse_id → warehouse.warehouse(id) – ship-from<br>created_by → auth.app_user(id)       |
| <b>so_line</b>     | id | so_id → sales.sales_order(id)<br>sku_id → catalog.sku(id)<br>uom_id → uom.unit(id)         |
| <b>allocation</b>  | id | so_line_id → sales.so_line(id)<br>bin_id → warehouse.bin(id)<br>lot_id → inventory.lot(id) |
| <b>backorder</b>   | id | so_line_id → sales.so_line(id)                                                             |

## Stock Movement Service (schema: movement)

| Table              | PK | Foreign keys → target                                                                                                                                                    |
|--------------------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>movement</b>    | id | sku_id → catalog.sku(id)<br>from_bin_id → warehouse.bin(id)<br>to_bin_id → warehouse.bin(id)<br>reason_code_id → movement.reason_code(id)<br>user_id → auth.app_user(id) |
| <b>transfer</b>    | id | from_warehouse_id → warehouse.warehouse(id)<br>to_warehouse_id → warehouse.warehouse(id)<br>sku_id → catalog.sku(id)                                                     |
| <b>adjustment</b>  | id | sku_id → catalog.sku(id)<br>bin_id → warehouse.bin(id)<br>reason_code_id → movement.reason_code(id)                                                                      |
| <b>reason_code</b> | id | (no foreign keys — this is a root table)                                                                                                                                 |

## Receiving Service (schema: receiving)

| Table              | PK | Foreign keys → target                                                                          |
|--------------------|----|------------------------------------------------------------------------------------------------|
| <b>asn</b>         | id | po_id → procurement.purchase_order(id)<br>supplier_id → supplier.supplier(id)                  |
| <b>asn_line</b>    | id | asn_id → receiving.asn(id)<br>po_line_id → procurement.po_line(id)<br>sku_id → catalog.sku(id) |
| <b>receipt</b>     | id | asn_id → receiving.asn(id)<br>warehouse_id → warehouse.warehouse(id)                           |
| <b>putaway</b>     | id | receipt_id → receiving.receipt(id)<br>bin_id → warehouse.bin(id)                               |
| <b>discrepancy</b> | id | receipt_id → receiving.receipt(id)<br>asn_line_id → receiving.asn_line(id)                     |
| <b>qa_check</b>    | id | receipt_id → receiving.receipt(id)<br>inspector_id → auth.app_user(id)                         |

## Fulfillment / Shipping Service (schema: fulfillment)

| Table            | PK | Foreign keys → target                                                                                    |
|------------------|----|----------------------------------------------------------------------------------------------------------|
| <b>pick_list</b> | id | so_id → sales.sales_order(id)<br>picker_id → auth.app_user(id)<br>warehouse_id → warehouse.warehouse(id) |
| <b>pack_slip</b> | id | pick_list_id → fulfillment.pick_list(id)                                                                 |
| <b>shipment</b>  | id | pack_slip_id → fulfillment.pack_slip(id)<br>carrier_id → fulfillment.carrier(id)                         |
| <b>carrier</b>   | id | (no foreign keys — this is a root table)                                                                 |
| <b>tracking</b>  | id | shipment_id → fulfillment.shipment(id)                                                                   |

## Replenishment Service (schema: replenishment)

| Table                 | PK | Foreign keys → target                                                                                     |
|-----------------------|----|-----------------------------------------------------------------------------------------------------------|
| <b>reorder_policy</b> | id | sku_id → catalog.sku(id)<br>warehouse_id → warehouse.warehouse(id)                                        |
| <b>forecast</b>       | id | sku_id → catalog.sku(id)<br>warehouse_id → warehouse.warehouse(id)                                        |
| <b>suggestion</b>     | id | sku_id → catalog.sku(id)<br>supplier_id → supplier.supplier(id)<br>warehouse_id → warehouse.warehouse(id) |

## Audit / Counting Service (schema: audit)

| Table                   | PK | Foreign keys → target                                                                                    |
|-------------------------|----|----------------------------------------------------------------------------------------------------------|
| <b>count_session</b>    | id | warehouse_id → warehouse.warehouse(id)<br>zone_id → warehouse.zone(id)<br>counter_id → auth.app_user(id) |
| <b>variance</b>         | id | session_id → audit.count_session(id)<br>sku_id → catalog.sku(id)<br>bin_id → warehouse.bin(id)           |
| <b>count_approval</b>   | id | variance_id → audit.variance(id)<br>approver_id → auth.app_user(id)                                      |
| <b>count_adjustment</b> | id | variance_id → audit.variance(id)<br>ledger_entry_id → inventory.stock_ledger(id)                         |

## UoM / Conversion Service (schema: uom)

| Table             | PK | Foreign keys → target                                    |
|-------------------|----|----------------------------------------------------------|
| <b>unit</b>       | id | (no foreign keys — this is a root table)                 |
| <b>conversion</b> | id | from_unit_id → uom.unit(id)<br>to_unit_id → uom.unit(id) |
| <b>barcode</b>    | id | sku_id → catalog.sku(id)<br>unit_id → uom.unit(id)       |

## Notification Service (schema: notification)

| Table             | PK | Foreign keys → target                                                                                                           |
|-------------------|----|---------------------------------------------------------------------------------------------------------------------------------|
| <b>alert</b>      | id | template_id → notification.template(id)<br>channel_id → notification.channel(id)<br>subscriber_id → notification.subscriber(id) |
| <b>template</b>   | id | (no foreign keys — this is a root table)                                                                                        |
| <b>channel</b>    | id | (no foreign keys — this is a root table)                                                                                        |
| <b>subscriber</b> | id | user_id → auth.app_user(id)                                                                                                     |

## Auth / User / Role Service (schema: auth)

| Table                   | PK | Foreign keys → target                                                 |
|-------------------------|----|-----------------------------------------------------------------------|
| <b>app_user</b>         | id | <i>(no foreign keys — this is a root table)</i>                       |
| <b>role</b>             | id | <i>(no foreign keys — this is a root table)</i>                       |
| <b>permission</b>       | id | <i>(no foreign keys — this is a root table)</i>                       |
| <b>user_role</b>        | id | user_id → auth.app_user(id)<br>role_id → auth.role(id)                |
| <b>role_permission</b>  | id | role_id → auth.role(id)<br>permission_id → auth.permission(id)        |
| <b>warehouse_access</b> | id | user_id → auth.app_user(id)<br>warehouse_id → warehouse.warehouse(id) |

## Reporting / Analytics Service (schema: reporting)

| Table                  | PK | Foreign keys → target                                               |
|------------------------|----|---------------------------------------------------------------------|
| <b>report_def</b>      | id | <i>(no foreign keys — this is a root table)</i>                     |
| <b>metric_snapshot</b> | id | report_id → reporting.report_def(id)                                |
| <b>export_job</b>      | id | report_id → reporting.report_def(id)<br>user_id → auth.app_user(id) |

## Pricing Service (schema: pricing)

| Table                   | PK | Foreign keys → target                                              |
|-------------------------|----|--------------------------------------------------------------------|
| <b>price_book</b>       | id | <i>(no foreign keys — this is a root table)</i>                    |
| <b>price_book_entry</b> | id | price_book_id → pricing.price_book(id)<br>sku_id → catalog.sku(id) |
| <b>cost</b>             | id | sku_id → catalog.sku(id)                                           |
| <b>margin</b>           | id | sku_id → catalog.sku(id)                                           |
| <b>discount_rule</b>    | id | price_book_id → pricing.price_book(id)                             |

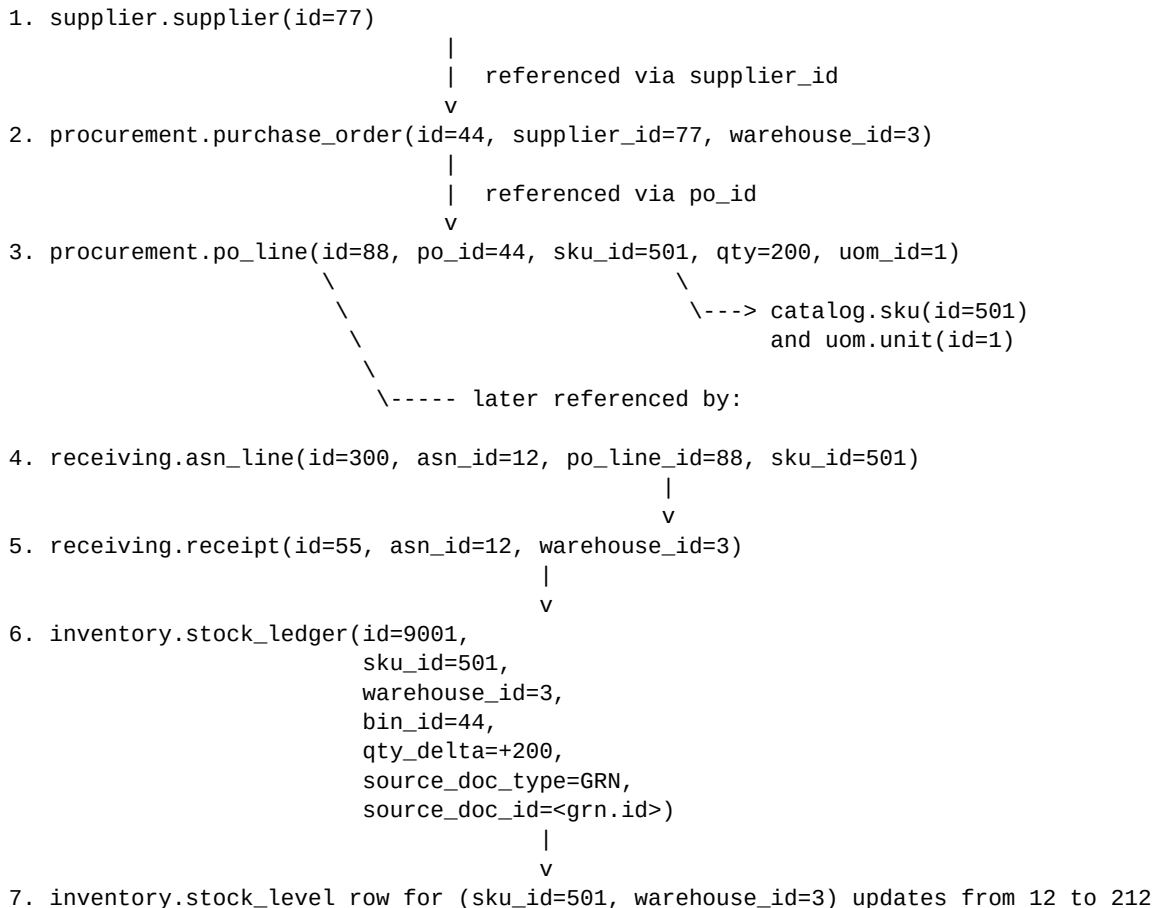


## Part 4 — Connection walkthroughs

Three end-to-end stories, written as a chain of primary-key/foreign-key links, so you can see the connections come to life with concrete row values.

### 4.1 The journey of an SKU through a purchase cycle

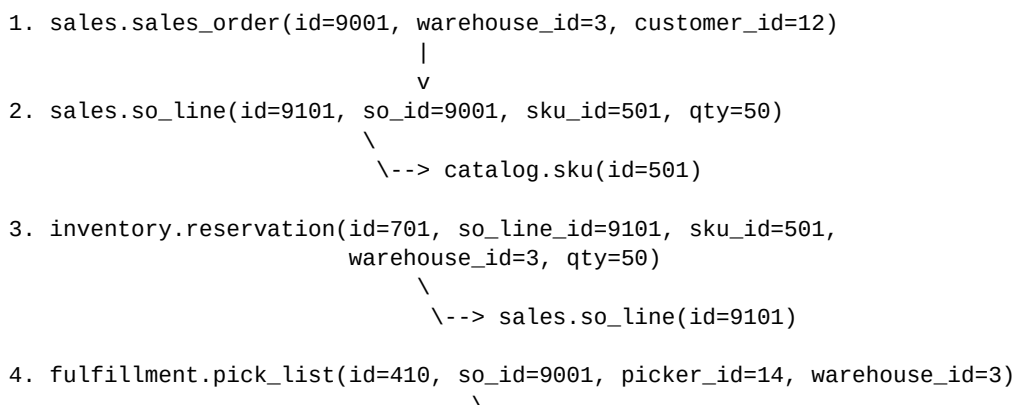
A buyer creates PO-2026-0044 for 200 units of SKU 501 from supplier 77. The chain of foreign keys generated by this single action:



Every arrow above is a foreign-key constraint enforced by Postgres. If any link in the chain is missing — say someone tries to insert a receipt for an ASN that does not exist — the database rejects the write.

### 4.2 The journey of an SKU through a sales cycle

A customer orders 50 units of SKU 501. The system needs to reserve stock, pick it, pack it, and ship it. Each step adds another row that points back at earlier rows by FK.



```

\--> auth.app_user(id=14)

5. inventory.stock_ledger(id=9050, sku_id=501, warehouse_id=3,
    qty_delta=-50, source_doc_type=PICK_LIST,
    source_doc_id=410)

6. fulfillment.pack_slip(id=611, pick_list_id=410)
    \
    \--> fulfillment.pick_list(id=410)

7. fulfillment.shipment(id=712, pack_slip_id=611, carrier_id=2)

8. fulfillment.tracking(id=812, shipment_id=712, waybill="DHL-XYZ-99")

```

### 4.3 A cycle count that finds a variance

An audit finds the physical count is 3 lower than the ledger. The correction must close the gap by writing a ledger row that the variance points at.

```

1. audit.count_session(id=22, warehouse_id=3, zone_id=7, counter_id=14)
    \
    \--> warehouse.zone(id=7)

2. audit.variance(id=88, session_id=22, sku_id=501, bin_id=44,
    expected_qty=212, counted_qty=209, diff=-3)

3. audit.count_approval(id=15, variance_id=88, approver_id=9)
    \
    \--> auth.app_user(id=9)

4. inventory.stock_ledger(id=9100, sku_id=501, warehouse_id=3,
    qty_delta=-3, source_doc_type=COUNT_ADJ,
    source_doc_id=22)

5. audit.count_adjustment(id=33, variance_id=88, ledger_entry_id=9100)
    \
    \--> the ledger row above

```

Notice how the loop closes: the ledger row is created first, then the `audit.count_adjustment` row points back at it. This means you can query "show me the ledger entry that corrected variance #88" with a single join.

## Part 5 — How this looks in SQL

### 5.1 Defining a primary key

```
CREATE TABLE catalog.sku (  
  id          BIGSERIAL PRIMARY KEY,  
  sku_code    TEXT          UNIQUE NOT NULL,  
  product_id  BIGINT        NOT NULL REFERENCES catalog.product(id),  
  uom_id      BIGINT        NOT NULL REFERENCES uom.unit(id),  
  name        TEXT          NOT NULL,  
  active      BOOLEAN       NOT NULL DEFAULT TRUE,  
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now()  
);
```

Three keys at play: `id` is the PK, `product_id` and `uom_id` are FKs that reach into other schemas, and `sku_code` is a UNIQUE business key (different from the PK).

### 5.2 Declaring foreign keys across schemas

```
CREATE TABLE procurement.po_line (  
  id          BIGSERIAL PRIMARY KEY,  
  po_id       BIGINT        NOT NULL  
              REFERENCES procurement.purchase_order(id)  
              ON DELETE CASCADE,  
  sku_id      BIGINT        NOT NULL REFERENCES catalog.sku(id),  
  uom_id      BIGINT        NOT NULL REFERENCES uom.unit(id),  
  qty_ordered NUMERIC(18,4) NOT NULL CHECK (qty_ordered > 0),  
  unit_price  NUMERIC(18,2) NOT NULL,  
  UNIQUE (po_id, sku_id)  
);
```

Three FKs declared. `ON DELETE CASCADE` means deleting a PO automatically deletes its lines. The other two FKs (`sku`, `uom`) do NOT cascade — you should not be able to delete a SKU while POs still reference it.

### 5.3 A junction table for many-to-many

```
CREATE TABLE auth.user_role (  
  id          BIGSERIAL PRIMARY KEY,  
  user_id     BIGINT NOT NULL REFERENCES auth.app_user(id) ON DELETE CASCADE,  
  role_id     BIGINT NOT NULL REFERENCES auth.role(id)      ON DELETE CASCADE,  
  granted_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (user_id, role_id)  
);
```

### 5.4 A query that uses six tables across five schemas

This single query answers "show me everything about purchase order 44". Because every relationship is wired by a foreign key, the joins are trivial.

```
SELECT  po.id          AS po_id,  
        po.po_number,  
        po.status,  
        s.name        AS supplier_name,  
        wh.name       AS ship_to_warehouse,  
        sku.sku_code,  
        sku.name      AS sku_name,  
        line.qty_ordered,  
        line.unit_price,  
        grn.grn_number,  
        grn.received_at  
FROM    procurement.purchase_order po  
JOIN    procurement.po_line  line ON line.po_id = po.id  
JOIN    catalog.sku          sku  ON sku.id    = line.sku_id  
JOIN    supplier.supplier    s    ON s.id      = po.supplier_id
```

```

JOIN    warehouse.warehouse wh    ON wh.id          = po.warehouse_id
LEFT JOIN procurement.grn    grn    ON grn.po_id      = po.id
WHERE   po.id = 44;

```

This is the query you literally could not write if every service had its own database. The fact that schemas live inside one Postgres instance is what makes it possible.

## 5.5 A query that uses the stock ledger

The ledger is so central it deserves its own example. "What is the current quantity of SKU 501 in Warehouse 3?" is just a SUM:

```

SELECT  sku_id,
        warehouse_id,
        SUM(qty_delta) AS on_hand
FROM    inventory.stock_ledger
WHERE   sku_id = 501 AND warehouse_id = 3
GROUP BY sku_id, warehouse_id;

```

"Show me the entire history of how that quantity changed":

```

SELECT  l.occurred_at,
        l.qty_delta,
        l.source_doc_type,
        l.source_doc_id,
        u.email AS user_email,
        l.notes
FROM    inventory.stock_ledger l
LEFT JOIN auth.app_user u ON u.id = l.user_id
WHERE   l.sku_id = 501 AND l.warehouse_id = 3
ORDER BY l.occurred_at;

```

## Part 6 — Rules of the road

A short list of conventions to follow without exception. Each one prevents a category of bug that haunts inventory systems.

### Every table has an id **BIGSERIAL PRIMARY KEY**

No composite PKs at the row level. If you need uniqueness across columns (e.g. one price entry per (price\_book, sku) pair) add a UNIQUE constraint, not a composite PK. This keeps foreign keys simple — every FK is a single BIGINT column.

### Every FK column is named `_id`

Read the column name and you know exactly what it points at. `sku_id` always points at `catalog.sku(id)`. No exceptions, no abbreviations.

### Cross-schema FKs are spelled with the schema prefix

`REFERENCES catalog.sku(id)`, not just `REFERENCES sku(id)`. Makes ownership visible in the DDL.

### Cascade only inside a single aggregate

Deleting a `purchase_order` cascades to its `po_lines` (they are part of the same document). But deleting a `sku` does NOT cascade — you should never be able to delete a SKU that has historical movements.

### Every operational row carries `created_at` and (when meaningful) `updated_at + user_id`

You should always be able to answer "who did this and when?" without leaving the table.

### Reservations point at the SO line that caused them

When the SO is cancelled, the reservation goes with it. Without that FK link, you end up with orphan reservations that lock stock forever — a classic inventory bug.

### The stock ledger is the only writer of quantity changes

Every workflow (receiving, picking, transfer, audit) writes a ledger row. `stock_level` is a materialized projection of the ledger, never updated directly.

### Read-only access across schemas is fine; write access is not

A service may SELECT from any schema. It may only INSERT/UPDATE/DELETE in its own. Enforce this with Postgres role grants, not with code review.

## Closing thought

Primary keys identify. Foreign keys connect. Together they turn a pile of 40 tables into one inventory system where every quantity, every order, every shipment, and every count adjustment can be traced back to the SKU and warehouse it concerns. Get the keys right and the rest of the system almost writes itself.